Source-to-Target Mapping (STM)

Retail Customer Intelligence Platform — full field-level lineage from the raw e-commerce CSV through the Medallion layers (Bronze → Silver → dbt staging → intermediate → Gold star schema + RFM) and into the ML churn mart.

`AIO Conquer 2026 · Module 01` — keep this in sync with the code; every rule below is implemented in the referenced file.

---

0. Layer Map & Owners

#	Hop	Engine / File	Grain	Materialization
1	Raw CSV → **Bronze**	`src/etl/bronze_ingest.py`	transaction line	Parquet, `year_month=` partitions
2	Bronze → **Silver**	`src/etl/silver_transform.py`	transaction line	Parquet, `year_month=` partitions
3	Silver → **stg**	`dbt/models/staging/stg_silver__transactions.sql`	transaction line	view
4	stg → **int**	`dbt/models/intermediate/int_transactions__prepared.sql`	transaction line	view
5	int → **Gold dims/fact**	`dbt/models/marts/*.sql`	dim = entity, fact = line	table
6	int → **mart_rfm**	`dbt/models/marts/mart_rfm.sql`	customer	table
7	Silver → **mart_churn_scores**	`ml/features.py` + `ml/score.py`	customer	Parquet (Gold)

> **Date rebasing happens once, in Bronze.** `original_invoice_date` (source) is shifted by **+5,295 days** (`2011-12-09` → `2026-06-10`) into `invoice_date`. Every downstream layer works on the shifted date; the original is carried for audit only.

---

1. Source Schema — `data/raw/online_retail_listing.csv`

Semicolon-delimited (`;`), `ISO-8859-1`, **European decimals** (comma), `dd.mm.yyyy HH:MM` dates, ~1.01M rows. All columns read as **string** first (`dtype=str`).

Source column	Example	Notes
`Invoice`	`489434`, `C489449`	`C` prefix = cancellation/return
`StockCode`	`85048`, `DOT`	product code
`Description`	`15CM CHRISTMAS GLASS BALL`	free text, may be blank
`Quantity`	`12`, `-6`	comma decimals possible; negative on cancellations
`InvoiceDate`	`01.12.2010 08:26`	`dd.mm.yyyy`, parsed `dayfirst`
`Price`	`6,95`	comma decimal → period
`Customer ID`	`13085`, *(blank)*	blank = guest/unknown
`Country`	`United Kingdom`	ship-to country

---

2. Raw CSV → Bronze · `src/etl/bronze_ingest.py`

Output schema `BRONZE_SCHEMA`. Rules applied in `normalize_columns()`.

Target (Bronze)	Type	Source	Transformation rule
`invoice`	string	`Invoice`	`strip()`
`stock_code`	string	`StockCode`	`strip().upper()` (avoid case-split)
`description`	string	`Description`	`strip()` (blank kept here, fixed in Silver)
`quantity`	float64	`Quantity`	`","→".", to_numeric(errors=coerce)`
`price`	float64	`Price`	`","→".", to_numeric(errors=coerce)`
`customer_id`	string	`Customer ID`	`NaN→"", strip()`
`country`	string	`Country`	`strip()`
`original_invoice_date`	timestamp	`InvoiceDate`	`to_datetime(format=mixed, dayfirst=True, errors=coerce)`
`invoice_date`	timestamp	`InvoiceDate`	`original_invoice_date + 5295 days` (rebase to ~2026)
`is_cancellation`	bool	`Invoice`	**derived:** `invoice.startswith("C")`
`ingested_at`	timestamp	—	**derived:** ingest run UTC timestamp (audit)
(partition) `year_month`	string	`invoice_date`	**derived:** shifted `YYYY-MM` (hive partition key)

3. Bronze → Silver · `src/etl/silver_transform.py`

Output schema `SILVER_SCHEMA`. Passthrough columns keep Bronze values; new columns below.

3.1 Cleaning (`run_cleaning`)

Target (Silver)	Source (Bronze)	Transformation rule
`invoice`, `stock_code`, `country`	same	`strip()` (re-asserted)
`description`	`description`	`strip()`; `"" → "UNKNOWN"`
`quantity`, `price`	same	`to_numeric(errors=coerce)` (type re-enforce)
`is_cancellation`	same	`astype(bool)`
`line_amount`	`quantity`, `price`	**derived:** `quantity * price`
(all rows)	—	**dedup:** `drop_duplicates()` on all columns
(row filter)	—	**drop noise:** non-cancellation rows with `price <= 0`

3.2 Derived calendar (`run_derive_calendar`, from shifted `invoice_date`)

Target (Silver)	Type	Rule
`invoice_year`	int32	`dt.year`
`invoice_month`	int32	`dt.month`
`invoice_day`	int32	`dt.day`
`invoice_quarter`	int32	`dt.quarter`
`invoice_day_of_week`	int32	`dt.dayofweek` (0 = Monday)
`invoice_week`	int32	`dt.isocalendar().week`
`year_month`	string	`dt.strftime("%Y-%m")`

> A per-partition `quality_report.json` + cumulative `_quality_log.jsonl` are written alongside the Parquet (`TabularDataQuality`), not part of the data schema.

---

4. Silver → dbt staging · `stg_silver__transactions.sql`

1:1 explicit-typed projection of the Silver Parquet (`read_parquet('../data/silver/...')`). No re-cleaning. All Silver columns are `CAST` to explicit types (e.g. `quantity → DOUBLE`, `invoice_date → TIMESTAMP`, `is_cancellation → BOOLEAN`). Column names unchanged.

---

5. dbt staging → intermediate · `int_transactions__prepared.sql`

`SELECT *` from staging **plus** two derived helpers used by the marts:

Target (int)	Type	Rule
`tx_date`	DATE	`CAST(invoice_date AS DATE)` — join key for `dim_date`
`is_valid_purchase`	BOOL	`customer_id <> '' AND is_cancellation = false AND line_amount > 0`

---

6. Intermediate → Gold Star Schema · `dbt/models/marts/`

6.1 `dim_customer` (grain: customer)

Target	Source	Rule
`customer_sk`	—	`ROW_NUMBER()`, **Unknown = 0** (empty `customer_id` sorted first)
`customer_id`	`int.customer_id`	business key; `''` row = Unknown/guest
`first_seen_date`	`int.invoice_date`	`MIN(invoice_date)` per customer (NULL for Unknown)
`segment`	—	placeholder `'Unknown'`/`'UNKNOWN'` (real segment in `mart_rfm`)

6.2 `dim_product` (grain: stock_code)

Target	Source	Rule
`product_sk`	—	`ROW_NUMBER() OVER (ORDER BY stock_code)`
`stock_code`	`int.stock_code`	distinct business key (upper-cased in Bronze)
`description`	`int.description`	`FIRST(description)` per `stock_code` (pick one)

6.3 `dim_country` (grain: country)

Target	Source	Rule
`country_sk`	—	`ROW_NUMBER() OVER (ORDER BY country)`
`country_name`	`int.country`	distinct country

6.4 `dim_date` (grain: calendar date)

Target	Source	Rule
`date_sk`	—	`ROW_NUMBER() OVER (ORDER BY dt)`
`date`	`int.tx_date`	distinct `tx_date`
`year` / `month` / `week` / `day_of_week` / `quarter`	`date`	`YEAR/MONTH/WEEK/DAYOFWEEK/QUARTER(dt)`

6.5 `fact_transactions` (grain: transaction line — purchases **and** cancellations)

Target	Source	Rule
`transaction_sk`	—	`ROW_NUMBER()` (degenerate PK)
`customer_sk`	`dim_customer`	`LEFT JOIN ON customer_id`, `COALESCE(..,0)` → Unknown
`product_sk`	`dim_product`	`LEFT JOIN ON stock_code`
`country_sk`	`dim_country`	`LEFT JOIN ON country = country_name`
`date_sk`	`dim_date`	`LEFT JOIN ON tx_date = date`
`quantity`	`int.quantity`	measure (negative on cancellations)
`price`	`int.price`	measure
`line_amount`	`int.line_amount`	measure — **NET revenue** = `SUM(line_amount)` (returns subtract)
`is_cancellation`	`int.is_cancellation`	kept as explicit business flag

---

7. Intermediate → `mart_rfm` (grain: customer) · `mart_rfm.sql`

Built from `is_valid_purchase` rows only; `ref_date = MAX(invoice_date)` of valid purchases.

Target	Source	Rule
`customer_id`	`int.customer_id`	group key
`recency_days`	`invoice_date`	`DATE_DIFF('day', MAX(invoice_date), ref_date)`
`frequency`	`invoice`	`COUNT(DISTINCT invoice)`
`monetary`	`line_amount`	`SUM(line_amount)`, `HAVING SUM > 0`
`r_score`	`recency_days`	`NTILE(5) OVER (ORDER BY recency_days DESC)` (fewer days = 5)
`f_score`	`frequency`	`NTILE(5) OVER (ORDER BY frequency ASC)`
`m_score`	`monetary`	`NTILE(5) OVER (ORDER BY monetary ASC)`
`segment`	`r_score`,`f_score`	`CASE` rule → Champions / Loyal Customers / Potential Loyalists / New Customers / Promising / Cannot Lose Them / At Risk / About to Sleep / Hibernating / Lost / Need Attention

---

8. Silver → `mart_churn_scores` (grain: customer) · `ml/features.py` + `ml/score.py`

ML feature matrix is engineered in pandas from the **Silver** layer (31 features, see `FEATURE_COLUMNS` in `ml/config.py`: RFM core, behavioral, temporal, engagement, monetary velocity). XGBoost (`ml/train.py`) scores every customer.

Target	Source	Rule
`customer_id`	feature matrix	business key
`churn_probability`	model	XGBoost `predict_proba` (0–1)
`churn_flag`	`churn_probability`	`>= optimal_threshold` (F1-optimized, ≈0.31) → bool
`risk_tier`	`churn_probability`	`>=0.7 High` · `>=0.4 Medium` · else `Low`

> **Label (training only):** `churn = 1` if a customer made **no** purchase in the last `EVALUATION_WINDOW_DAYS = 90` days of the shifted timeline (see `docs/ml_document/ml_design.md`).

---

9. Key Reconciliation Rules (for QA / dbt tests)

- `fact_transactions` row count = `stg_silver__transactions` row count (no row loss in marts).

- `SUM(line_amount)` in fact reconciles with Bronze→Silver net revenue (`assert_rfm_monetary_reconcile*`).
- Every `mart_rfm.customer_id` exists in `dim_customer`; segment ∈ accepted label set.
- Bidirectional rule `quantity < 0 ⟺ is_cancellation` — **one known source anomaly**: invoice `C496350` ("Manual" adjustment, `quantity = +1`) violates it (tracked by `assert_fact_quantity_cancellation_consistency` / `assert_stg_negative_qty_price_rules`).